

One Semicolon.

Full Server RCE



I Was Treating Header Injection as Low-Impact.

You know the finding I mean.

Abhishek meena

Published August 12, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

How a git push actually works	4
The semicolon	5
Where it gets spicy	6
The GitHub.com twist	7
Why this should bother you	8
What I do now	8
The part I keep thinking about	9

You send a request, add a `\r\n`, and boom — your input shows up as a new header. Or you throw a `;` in a value and suddenly there's a second key-value pair staring back at you.

Cute, you think. Novelty at best. ☹️

You write it up. "Reflected header injection — low impact." Triage sets it to P4. You get \$50. ☐ You move on to something that actually pays.

I did this for years. Found it, filed it, forgot it. Header injection felt like a dead cat — everyone had reported it, everything had been found. 2019 was the interesting era. This was just noise.

Then Wiz Research dropped CVE-2026-3854 in March. ☐

One semicolon. In a git push option. Remote code execution on GitHub's servers. Not on some random startup — on GitHub.com. 150 million developers. CVSS 8.7. "One of the highest rewards in the history of our bug bounty program," GitHub's own CISO said.



And the entire chain started with a semicolon I would have filed as P4 and walked away from.

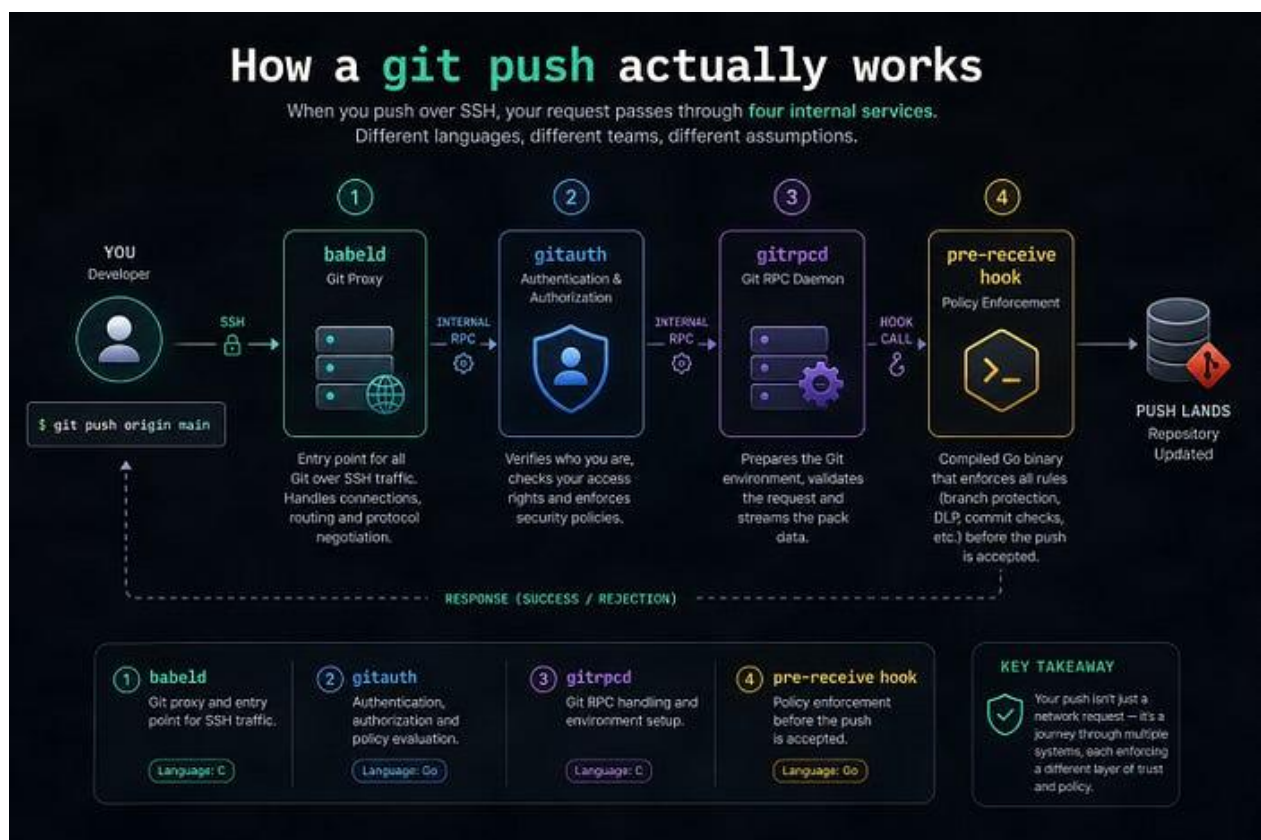
That report broke my brain a little. Not because the technique was new — it wasn't. Because of what it implies about every "minor" header injection I ever skipped.

Let me walk you through it. It's genuinely fun. ☐

How a git push actually works

When you `git push` over SSH, your request passes through four internal services. Different languages, different teams, different assumptions.

- **babeld** — the git proxy. Entry point. Your SSH connection lands here.
- **githash** — checks who you are, whether you can push, and what security policies apply to your session.
- **gitrpcd** — sets up the actual push environment.
- **pre-receive hook** — a compiled Go binary that enforces the rules before your push lands.



The interesting part is how they talk to each other.

babeld doesn't pass some clean JSON around. It builds an internal header called `X-Stat` — a flat string of `key=value` pairs separated by semicolons.

```
X-Stat: repo_id=12345; user_id=42; rails_env=production;  
large_blob_rejection_enabled=bool:true
```

This header carries security-critical stuff. The environment. The size limits. The hook config. And it gets parsed downstream by splitting on ;.

gitrpcd doesn't authenticate anything. It doesn't second-guess anything. It reads `X-Stat` and treats every field as the truth. Because why wouldn't it? This header was built by babeld. By their own service. By the trusted side.

That's the assumption that killed them. ☐

The semicolon

Now — git has a feature called push options. `git push -o` any string you want. It's meant for "hey server, here's a hint about this push."

```
git push -o "ci.skip" origin main
```

Any user can push any string they want. That's the feature.

babeld copies those push option values into `X-Stat` as `push_option_0`, `push_option_1`, and so on.

Verbatim. No sanitization. ☐

Forward the semicolon, and you've broken out of your field. The parser splits on ;, remember. Your push option value isn't one value anymore — it's new fields.

```
git push -o "x;large_blob_rejection_enabled=bool:false" origin main
```

That becomes:

```
X-Stat: ...; large_blob_rejection_enabled=bool:true; ...;  
push_option_0=x;large_blob_rejection_enabled=bool:false;  
push_option_count=1; ...
```

When gitrpcd splits on ;, it sees two different values for the same key. Which one wins?

Last one. ☹

The map is last-write-wins. Your injected `bool:false` appears later in the header, so it silently overrides the legitimate `bool:true`.

Wiz confirmed this on a live GHES instance with a packet capture. The injected fields appeared right next to the real ones, overriding them. No ambiguity. No race. Just a semicolon doing exactly what semicolons do to flat delimited formats.

If you stopped there — "I can inject a field into an internal header" — that's the P4 I used to file.

Where it gets spicy

Wiz didn't stop there. They reverse-engineered the pre-receive hook binary — a compiled Go binary, closed source — and mapped every X-Stat field to what it actually controls.

That's where the impact lives.

Field	What it controls
<code>rails_env</code>	Which execution path the hooks take — sandboxed, or bare metal
<code>custom_hooks_dir</code>	Where it looks for hook scripts
<code>repo_pre_receive_hooks</code>	The actual hook definitions, including script paths
<code>large_blob_rejection_enabled</code>	File size limits (mild, boring)

Here's the thing about `rails_env`. The pre-receive binary has two paths. If the value is `production`, hooks run inside a sandbox. Restricted. Contained.

Anything else?

Straight into the open. ☹️ No sandbox. No isolation. Runs as the `git` user with full filesystem access.

The entire security boundary between those two paths is one string value. One string value you can now overwrite.

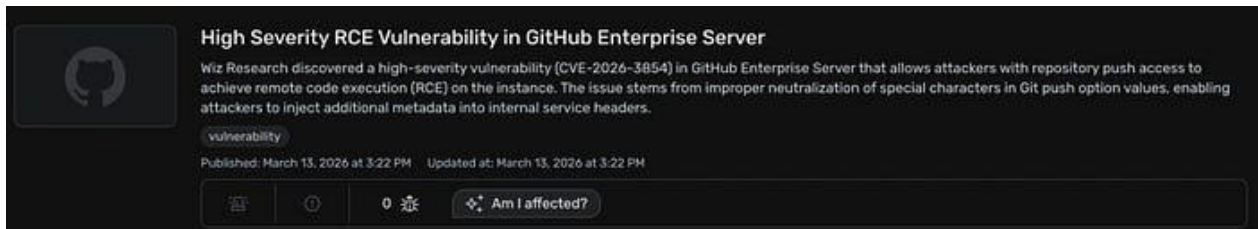
So they flipped it.

`rails_env` → not production. Sandbox gone.

Then `custom_hooks_dir` → point it wherever you want. Then `repo_pre_receive_hooks` → a JSON hook definition whose "script" field says `../../../../../../../../bin/id`. Path traversal. The binary resolves it, joins it to your controlled directory, and runs an arbitrary binary as the git user.

One push:

```
git push -o "x;rails_env=development;custom_hooks_dir=/tmp/
attacker;repo_pre_receive_hooks=[{"hook_id":1,"name":"pwn","script":"../../../../../
../../bin/id"}]" origin main
remote: uid=500(git) gid=500(git) groups=500(git)      ← hello from the server
To github.com:user/repo.git
abc1234..def5678  master -> master
```



RCE, full server compromise, as the git service user. Every repo on that server. Every secret in its config. □

From a semicolon.

The GitHub.com twist

Now here's my favorite part. $\square$

The chain worked on GitHub Enterprise Server. They pointed it at GitHub.com next. And... nothing. Push completes, no hooks run, no code exec. Dead end.

Not deterred. They injected `user_operator_mode=bool:true` — a debug flag — to see what was happening. Side by side, GHES vs GitHub.com. The custom-hooks path just wasn't being reached on GitHub.com.

So they went back into the binary. Found a boolean `X-Stat` flag that gates whether the server operates in enterprise mode. On GHES it defaults to true. On GitHub.com it defaults to false, so the custom-hooks path never fires.

And that flag... also lived in X-Stat . Also injectable through the same semicolon. 🐱

One more injected field. Same chain. This time they ran `hostname:`

```
remote: starting PreReceiveCustomHooksCheck hook...
remote: [REDACTED].github.net ← inside GitHub.com
remote: finished PreReceiveCustomHooksCheck hook in 2ms
```

RCE on GitHub.com. $\square$

And here's the part that should make you sit up straight. The git user on a GitHub.com storage node serves all repositories on that node. Millions of repos, every org, every user. They enumerated indexes on the compromised nodes — millions of entries, other people's code, other companies' secrets.

They didn't touch any of it. Validated with their own test accounts. But the capability was sitting right there. That's the ceiling of this "minor" bug.

GitHub fixed it in 75 minutes — percent-encode the semicolons at the boundary. One line. ✂ And removed a code path from production that should never have been there.

The fix was trivial. Because the bug was never hard. It was just hiding. □

Why this should bother you

Stop and think about what those researchers did differently from most of us.

They didn't find a fancy bug. They found a semicolon. The exact bug I used to classify as P4 novelty and delete from my notes. □

The difference was what they did after the semicolon.

They followed the field. Through the internal header. Into a compiled binary they reverse-engineered. To a list of keys that control security decisions. To an execution path that should never be reachable in production. And they chained one tiny primitive into full server compromise.

The impact wasn't at the injection point. It was four hops deep, on the other side of a trust boundary. From where you're standing — outside the internal network — you literally cannot see that impact. The header vanishes into the backend. You only know it landed if you chased it.

That's why we all file these as P4. Not because they're P4. Because we never look past what we can see.

This is the same skeleton as request smuggling. As CRLF injection. As cookie tossing. A builder serializes data into a flat string, a parser interprets it with different rules, and somewhere in between, your input changed meaning.

The delimiter changes. The trust boundary changes. Last-write-wins stays.

What I do now

I don't file every header injection as critical. That would be noise. I do spend ten extra minutes on every single one, asking:

What's in the same header? I stop looking at my injected value and start looking at everything else in that string. Is there an `is_admin`? A `rails_env`? A `sandbox` flag? A role? Those are the fields that matter. The injection is just the key in the door — the field you can overwrite is the room you reach.

Can I overwrite, or just add? Creating new fields is usually a party trick — the parser may ignore unknowns. Overwriting an existing security-critical value is the real signal. And if it's last-write-wins, your value appearing later wins silently. That silence is the point. No error. No log. Just your value, now the truth.

What decision does that field drive? Trace it. A logging flag is worthless. A field that picks between a sandbox and no sandbox is a kingdom. Names leak meaning — `env`, `mode`, `role`, `path_base`. Difference in behavior when you flip a value tells you even more.

Does it cross a trust boundary? The killer combination is: a service that sees attacker input building the string, and a service on the other side that assumes the string is trustworthy. That boundary is where "cute" becomes "critical." Ask who's parsing it downstream. What they assume. What they'd never defend against.

Is there a non-production path in production? This one's almost always there. Code that runs without sandboxing, without checks, without limits — shipped into the prod image because nobody noticed. If the field you control picks the execution path, someone left you a back door wearing a uniform.

Ten minutes. Five questions. □ That's all it takes to turn a \$50 finding into a real one. □

The part I keep thinking about

A year ago, if someone handed me this exact finding — semicolon in a push option → new field in an internal header — I'd have written "P4, internal impact, cosmetic" and moved on.

Wiz turned that into CVSS 8.7.

They didn't have a better payload. They had a better question, asked one hop deeper into the system. "What security decision does this field drive?"

That question is free. You can ask it on any bug you find tomorrow.

Next time you see a stray delimiter do something it shouldn't — don't file it. Chase it. See what it reaches. Ask what it controls. Follow it past the boundary where everything assumes it's harmless.

The semicolon was never the bug.

What it could reach was.

Sources:

- [Wiz Research — GitHub RCE Vulnerability: CVE-2026-3854 Breakdown](https://www.wiz.io/blog/github-rce-vulnerability-cve-2026-3854) (https://www.wiz.io/blog/github-rce-vulnerability-cve-2026-3854)
- [GitHub Security Blog — Securing the git push pipeline](https://github.blog/security/securing-the-git-push-pipeline-responding-to-a-critical-remote-code-execution-vulnerability/) (https://github.blog/security/securing-the-git-push-pipeline-responding-to-a-critical-remote-code-execution-vulnerability/)
- [GitHub Advisory GHSA-64fw-jx9p-5j24](https://github.com/advisories/GHSA-64fw-jx9p-5j24) (https://github.com/advisories/GHSA-64fw-jx9p-5j24)
- [NVD — CVE-2026-3854](https://nvd.nist.gov/vuln/detail/CVE-2026-3854) (https://nvd.nist.gov/vuln/detail/CVE-2026-3854)